Media.net OA

Problem 1: Magic Number Array

Problem Description

Given a permutation of size N in integer array A. For each number, find if the value of its magic number is ≥ 0 or negative. The value of a magic number is defined as: (count of smaller numbers to the left $\times$ count of smaller numbers to the right) $-$ (count of larger numbers to the left $\times$ count of larger numbers to the right).

Return an integer array where `ans[i]` is 1 if the magic number is ≥ 0 or -1 if it is negative.

Problem Constraints

$1 \leq N \leq 10^5$

C++ Solution

```cpp
vector<int> Solution::solve(vector<int>& A) {
    int n = A.size();
    vector<int> chk(n + 1, 0);

    auto add = [&](int i, int val) {
        while (i <= n) {
            chk[i] += val;
            i += i & (-i);
        }
    };

    auto get = [&](int i) {
        long long s = 0;
        while (i > 0) {
            s += chk[i];
            i -= i & (-i);
        }
        return s;
    };

    vector<int> answ;
    int i = 0;
    while (i < n) {
        int vy = A[i];
        long long slam = get(vy - 1);
        long long sram = vy - 1 - slam;
        long long lloo = i - slam;
        long long lrr = n - 1 - i - sram;

        long long m = (slam * sram) - (lloo * lrr);

        if (m >= 0)
            answ.emplace_back(1);
        else
            answ.emplace_back(-1);

```

```
37          add(vy, 1);
38          i++;
39      }
40
41      return answ;
42  }
```

Problem 2: Allied Nations' Tour

Problem Description

The allied nations in total have A cities numbered from 1 to A. Currently you are at city B. Your task is to take a tour of the given A cities for C days. From a city i you can go to any city in the range $i - E$ to $i + E$ and you cannot go out of range.

The intel core has provided M information given in the form of a 2D array D of size M. According to this, at day `D[i][0]` Akatsuki will attack on city `D[i][1]`. You cannot be present in some city on a certain day if Akatsuki will arrive there.

Find the total number of tours you will be able to take. Since the answer can be big, output it modulo 1000000007.

Problem Constraints

$1 \leq A, C \leq 1000$
$1 \leq M \leq A \times C$

C++ Solution

```cpp
int Solution::solve(int A, int B, int C, vector<vector<int> > &D, int E) {
    int mod = 1000000007;
    vector<vector<bool>> atk(C + 1, vector<bool>(A + 1, false));

    for (auto& x : D) {
        if (x[0] <= C && x[1] <= A) {
            atk[x[0]][x[1]] = true;
        }
    }

    vector<int> pd(A + 1, 0);
    pd[B] = 1;

    for (int d = 1; d <= C; ++d) {
        vector<int> pr(A + 1, 0);
        for (int c = 1; c <= A; ++c) {
            pr[c] = (pr[c - 1] + pd[c]) % mod;
        }

        vector<int> cur(A + 1, 0);
        for (int c = 1; c <= A; ++c) {
            if (!atk[d][c]) {
                int l = max(1, c - E);
                int r = min(A, c + E);
                cur[c] = (pr[r] - pr[l - 1] + mod) % mod;
            }
        }
        pd = cur;
    }

    int ans = 0;
    for (int c = 1; c <= A; ++c) {
        ans = (ans + pd[c]) % mod;
    }

    return ans;
}
```

Problem 3: Matrix 2x2 XOR Conversion

Problem Description

Given 2 matrices A and B of size $N \times N$, all the elements are either 0 or 1. We perform an operation on matrix A any number of times: choose any 2×2 submatrix of A and make each element `A[i][j] = A[i][j] XOR 1` for each (i, j) in this submatrix.

Check if it is possible to convert A into B. Return 1 if possible and 0 if not.

Problem Constraints

$2 \leq |A| = |B| \leq 10^3$

C++ Solution

```cpp
int Solution::solve(vector<vector<int> > &A, vector<vector<int> > &B) {
    int n = A.size();
    if (n == 0) return 1;

    int i = 0;
    while (i < n - 1) {
        int j = 0;
        while (j < n - 1) {
            if (A[i][j] != B[i][j]) {
                A[i][j] ^= 1;
                A[i+1][j] ^= 1;
                A[i][j+1] ^= 1;
                A[i+1][j+1] ^= 1;
            }
            j++;
        }
        i++;
    }

    for (int k = 0; k < n; k++) {
        if (A[n-1][k] != B[n-1][k] || A[k][n-1] != B[k][n-1]) {
            return 0;
        }
    }

    return 1;
}
```